

# ThreatGPT: A Hierarchical Multi-Agent Architecture for LLM-Based Cyber Threat Intelligence Analysis — Design, Preliminary Evaluation, and Open Challenges

[TODO: Author Name(s) – to be completed by the authors]

[TODO: Affiliation, City, Country]

[TODO: email@institution.edu]

**Abstract.** Cyber Threat Intelligence (CTI) analysts face an increasing volume and velocity of unstructured threat data that traditional signature-based and single-model machine-learning pipelines struggle to keep pace with. Large Language Models (LLMs) offer new opportunities for automating extraction, technique mapping, and risk assessment, but existing single-model approaches conflate distinct sub-tasks, rely on static knowledge, and rarely expose their reasoning to analysts. We present ThreatGPT, a hierarchical multi-agent architecture that decomposes LLM-based CTI analysis into three specialised roles — an Extractor Agent, a Mapper Agent that grounds extracted intelligence against the MITRE ATT&CK framework, and a Risk Assessor Agent — coordinated through a temporally-weighted retrieval-augmented knowledge layer and a set of adversarial-robustness safeguards. We describe the architecture, its prompting and retrieval design, and its intended evaluation protocol in detail. As a first, modest empirical step, we implement and evaluate a non-LLM TF-IDF retrieval baseline for the Mapper Agent’s core task — mapping free-text threat descriptions to MITRE ATT&CK techniques — against a hand-labelled set of 49 CTI sentences and the full current catalogue of 222 top-level Enterprise ATT&CK techniques. This baseline reaches 53.1% top-1 accuracy, 71.4% top-3 accuracy, and a Mean Reciprocal Rank of 0.655, substantially above a random-ranking baseline (MRR 0.027), giving a concrete, reproducible reference point against which the full LLM-based pipeline can later be measured. We discuss why we expect LLM- and RAG-based mapping to improve on this baseline, detail the remaining implementation and evaluation work required to validate the complete three-agent system, and are explicit about what has and has not yet been empirically demonstrated. We argue this staged, transparently-scoped approach is preferable to reporting speculative end-to-end numbers, and we release our baseline code and labelled evaluation set to support reproducible follow-on work.

**Keywords:** Cyber Threat Intelligence · Large Language Models · Multi-Agent Systems · MITRE ATT&CK · Retrieval-Augmented Generation · Adversarial Robustness

# 1 Introduction

## 1.1 Context and Motivation

Security Operations Centres (SOCs) are structurally overwhelmed: enterprise SOCs routinely process well over 100,000 alerts per day, and analysts report spending two to four hours per shift on manual triage, with a substantial fraction of alerts left uninvestigated [1]. The volume of new, publicly disclosed vulnerabilities has grown from a few thousand per year in 2016 to over 46,000 per year in 2025 [2], and open-source threat reporting continues to expand in parallel, far beyond what manual CTI teams can synthesise. At the same time, adversaries are themselves beginning to weaponise AI agents to plan and execute intrusions with reduced human involvement, a shift that recent threat-intelligence reporting shows can produce disproportionately high-risk campaigns without a correspondingly large technique footprint [3].

Traditional CTI automation – signature matching, rule-based IOC extraction, and supervised classifiers trained on hand-engineered features – struggles to keep pace with this landscape: such systems generalise poorly to novel phrasing, require constant retraining as the threat landscape evolves, and typically address only narrow sub-tasks of the CTI lifecycle [4]. LLMs have recently demonstrated strong performance on exactly the kind of unstructured-text understanding this domain requires, motivating a wave of recent work on LLM-assisted TTP (Tactics, Techniques, and Procedures) extraction, ATT&CK technique mapping, and agentic SOC automation [5,6,7,8].

## 1.2 Research Gap

Despite this progress, we identify three persistent limitations in the existing literature:

- **Limitation 1 – Monolithic, single-model approaches lack specialisation.** Most LLM-based CTI systems use a single model (often via zero- or few-shot prompting) to perform extraction, technique mapping, and risk interpretation simultaneously [7,9]. This conflates sub-tasks with very different failure modes – entity extraction is fundamentally an information-extraction problem, technique mapping is a large-label-space retrieval problem, and risk assessment requires organisational context that has no natural place in a single prompt.
- **Limitation 2 – Flat or static knowledge grounding.** Retrieval-based mapping systems such as TechniqueRAG embed and index the full, flat set of 200+ ATT&CK techniques uniformly, ignoring the framework’s hierarchical tactic/technique/sub-technique taxonomy and the fact that technique prevalence and adversary tooling shift over time [4]. Static knowledge bases used for extraction and mapping also become stale as new techniques (e.g., the recently added AI-service-abuse techniques T1682/T1683 in ATT&CK) are introduced.

- **Limitation 3 – Limited explainability and robustness.** LLM outputs in this domain are typically returned as final labels without exposed reasoning chains, and current systems are known to be vulnerable to prompt injection and RAG-poisoning attacks: as few as five adversarial documents in a retrieval corpus have been shown to manipulate RAG outputs in up to 90% of queries in controlled studies [16,17]. A CTI system that silently ingests attacker-controlled OSINT text is itself a plausible injection surface.

### 1.3 Our Contribution

This paper makes the following contributions:

1. A hierarchical multi-agent architecture, ThreatGPT, that decomposes LLM-based CTI analysis into an Extractor Agent, a Mapper Agent, and a Risk Assessor Agent, each with a distinct model role, prompt design, and evaluation protocol (Section 3).
2. A design for temporally-weighted, hierarchy-aware retrieval-augmented knowledge integration intended to address Limitation 2 (Section 3.5).
3. A set of adversarial-defense design principles for LLM-based CTI pipelines, informed by the current prompt-injection and RAG-poisoning literature (Section 3.6).
4. A concrete, reproducible *preliminary* baseline experiment for the Mapper Agent’s core sub-task – CTI-sentence-to-ATT&CK-technique retrieval – using the full, current MITRE ATT&CK Enterprise catalogue and a hand-labelled evaluation set, which we release together with our code (Section 4).
5. An explicit, honestly-scoped discussion of what remains to be implemented and empirically validated before the full three-agent ThreatGPT pipeline can be claimed to outperform existing single-model or non-agentic baselines (Section 5).

We emphasise contribution 5 because we believe it matters: a growing share of the LLM-for-security literature reports end-to-end numbers for systems whose components were not independently ablated or whose baselines were not run under comparable conditions. We instead report one real, reproducible number now, describe precisely what further engineering and evaluation is required, and treat the remainder of the pipeline as a concrete, falsifiable research and engineering programme rather than as an already-validated result.

### 1.4 Paper Organisation

Section 2 reviews CTI fundamentals, LLMs in security, RAG, and multi-agent systems. Section 3 presents the ThreatGPT architecture in full. Section 4 describes our preliminary baseline experiment and its results. Section 5 discusses practical implications, current limitations, and ethical considerations. Section 6 situates ThreatGPT relative to closely related systems. Section 7 concludes and outlines future work.

## 2 Background and Related Work

### 2.1 Cyber Threat Intelligence Fundamentals

CTI is commonly described as a lifecycle of collection, processing, analysis, and dissemination, in which raw indicators and unstructured reporting are progressively refined into actionable, decision-relevant knowledge [10]. Early automation efforts relied on signature matching and hand-authored extraction rules; later work applied classical supervised machine learning (e.g., Random Forests, SVMs) over hand-engineered lexical and syntactic features [11]. The MITRE ATT&CK framework has become the de facto standard vocabulary for describing adversary behaviour, organising techniques hierarchically under 14 tactics, and is now the near-universal grounding ontology for automated TTP-extraction research [12].

### 2.2 Large Language Models in Security

Transformer-based LLMs have been applied across a growing range of security tasks, from vulnerability description understanding to fully agentic penetration-testing frameworks [13]. Within CTI specifically, recent systems fine-tune or prompt LLMs for TTP extraction (IntelEx [5], CTINexus [6], AECR), for ATT&CK mapping (TechniqueRAG, Rule-ATT&CK Mapper [19], MITRE’s own TRAM tool [20]), and for broader knowledge-graph construction from OSINT under data scarcity [6]. A NeurIPS 2024 benchmark, CTIBench, was introduced specifically to standardise evaluation of LLMs on CTI sub-tasks [9]. Nonetheless, current systematic reviews of LLM-driven cyber defence – one recent PRISMA-guided review synthesising 144 SCI-indexed studies – describe the field as “rapidly expanding yet structurally imbalanced,” with disproportionate attention to detection and SOC optimisation relative to rigorous, comparable evaluation of CTI reasoning [14].

### 2.3 Retrieval-Augmented Generation

RAG grounds LLM generation in retrieved external documents, mitigating (but not eliminating) hallucination and static-knowledge staleness [15]. For ATT&CK mapping specifically, TechniqueRAG embeds the full technique catalogue and retrieves the nearest neighbours to a query passage [4]; recent work has identified that this “flat” retrieval design discards the taxonomy’s hierarchical structure, motivating hierarchy-aware retrieval variants [4]. Separately, a substantial body of recent security research documents that RAG pipelines are themselves attackable: adversarial or poisoned documents inserted into a retrieval corpus can manipulate downstream generation with a small number of crafted documents [16], and the OWASP Top 10 for LLM Applications (2025 revision) now lists vector/embedding weaknesses and RAG-specific risks as first-class categories [17].

## 2.4 Multi-Agent Systems

Decomposing complex LLM tasks across multiple specialised, communicating agents has become a common design pattern (e.g., AutoGPT- and MetaGPT-style frameworks), and has been applied specifically to SOC workflows: AgentSOC integrates hypothesis generation, structural validation against enterprise topology, and risk-aware action scoring in a closed loop [1]; a recent industrial case study of a telecommunications-provider SOC similarly reports moving from single-model helpers toward distributed multi-agent CTI pipelines to handle the scale of modern OSINT and vulnerability disclosure volumes [2]. Multi-agent penetration-testing frameworks (Incalmo, MAPTA) show comparable specialisation benefits on the offensive side [13]. A recurring theme across this literature is that multi-agent decomposition improves task specialisation and auditability, but introduces new coordination, consistency, and inter-agent trust challenges that single-agent systems do not face [14].

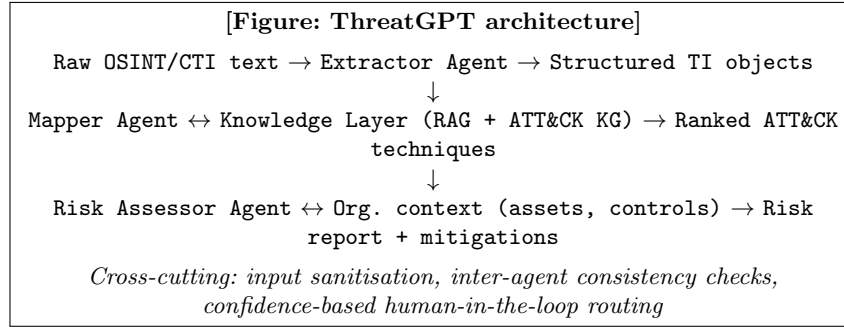
## 2.5 Threat Intelligence Automation and Positioning

Existing automated CTI tools range from open community tooling (MISP, the TRAM mapper [20]) to commercial threat-intelligence platforms, and from purely extractive systems to end-to-end agentic pipelines. Relative to this landscape, ThreatGPT’s contribution is architectural: an explicit three-stage decomposition (extraction  $\rightarrow$  mapping  $\rightarrow$  risk assessment) with a shared, temporally-weighted, hierarchy-aware knowledge layer and adversarial safeguards designed in from the outset, rather than retrofitted. We position this as a design and preliminary-validation contribution, distinct from papers that report full end-to-end evaluation numbers; Section 6 provides a structured qualitative comparison.

# 3 Proposed Methodology: ThreatGPT

## 3.1 System Architecture Overview

Figure 1 (described textually below, given the constraints of this document) depicts the ThreatGPT pipeline as a directed flow: raw inputs (OSINT feeds, CTI reports, social media, internal SOC tickets) enter the *Extractor Agent*, which emits structured intelligence objects (entities, relations, candidate IOCs, and free-text TTP descriptions). These objects are passed to the *Mapper Agent*, which grounds each TTP description against the MITRE ATT&CK taxonomy via the shared knowledge layer, producing ranked technique candidates with confidence scores. The *Risk Assessor Agent* consumes the mapped techniques together with organisational asset and control context to produce a prioritised risk report with recommended mitigations. A cross-cutting *consistency and defense layer* monitors all three agents’ inputs and outputs for injected instructions, out-of-distribution inputs, and inter-agent disagreement, and can route low-confidence cases to a human analyst.



**Fig. 1.** High-level ThreatGPT architecture. Each stage is a distinct agent with its own model, prompt design, and evaluation protocol; a cross-cutting defense layer inspects all agent boundaries.

### 3.2 Extractor Agent

**Role.** Convert raw, unstructured text into structured threat-intelligence objects: named entities (malware, threat actors, CVEs, infrastructure indicators), relations between them, and short free-text TTP descriptions suitable for downstream mapping.

**Design.** We propose a fine-tuned open-weight model (e.g., an 8B-parameter LLaMA-3-class model via parameter-efficient fine-tuning such as LoRA/QLoRA) rather than a proprietary API model, to allow on-premises deployment of raw, potentially sensitive OSINT and internal ticket text. Candidate fine-tuning corpora include the TRAM-annotated report corpus [20] and other publicly released CTI-annotation datasets used in recent extraction work [6,5].

**Illustrative prompt template (extraction sub-task):**

System: You are a CTI extraction assistant. Given a threat report excerpt, extract: (1) named entities with type labels [malware, threat-actor, CVE, infrastructure, tool]; (2) relations between entities; (3) short (<=25 word) behavioural statements describing observed adversary actions. Output strict JSON. Do not infer facts not present in the text.  
User: {report\_excerpt}

**Evaluation protocol (proposed).** Entity- and relation-level Precision, Recall, and F1 against a held-out annotated test split, following standard NER/RE evaluation practice; we have not yet fine-tuned or evaluated this agent (see Section 5).

### 3.3 Mapper Agent

**Role.** Given a short behavioural statement from the Extractor Agent, retrieve and rank the most plausible MITRE ATT&CK technique(s), with a calibrated confidence score.

**Design.** We propose an API-based frontier model (e.g., Claude or GPT-class) operating over a hierarchy-aware retrieval index, addressing Limitation 2 from Section 1: rather than TechniqueRAG’s flat embedding of all 200+ techniques [4], the index first narrows candidates by tactic, then by top-level technique, before ranking sub-techniques – reducing the effective label space the generative step must discriminate over.

**Confidence scoring.** We use temperature-scaled calibration (Equation 3) so that reported confidence approximates true correctness probability, following standard neural calibration practice.

**Preliminary, non-LLM baseline.** Section 4 reports results for a TF-IDF cosine-similarity retrieval baseline for exactly this sub-task – a reference point we expect a properly designed hierarchy-aware LLM+RAG mapper to exceed, but which we have not yet built.

### 3.4 Risk Assessor Agent

**Role.** Combine mapped techniques with organisational asset criticality and existing control coverage to produce a prioritised, human-readable risk report with mitigation recommendations aligned to NIST, CIS, or ISO 27001 control catalogues.

**Proposed risk scoring.** We propose a composite score combining likelihood, impact, and asset criticality:

$$R = L \times I \times C \times \alpha \quad (1)$$

where  $L \in [0, 1]$  is the estimated likelihood of the mapped technique being used against the organisation (informed by the Mapper Agent’s confidence and technique prevalence in the knowledge layer),  $I \in [0, 1]$  is estimated business impact if the technique succeeds,  $C \in [0, 1]$  is asset criticality, and  $\alpha$  is an organisation-specific calibration constant. We treat this formula, and the qualitative mitigation-mapping step, as design proposals; neither has been empirically validated against expert-assigned ground-truth risk rankings.

### 3.5 Dynamic Knowledge Integration

The knowledge layer shared by the Mapper and Risk Assessor agents is designed around three components:

**Temporal weighting.** Recency-weighted retrieval down-weights older threat-intelligence entries relative to newer ones, using exponential decay:

$$w(t) = e^{-\lambda t} \quad (2)$$

where  $t$  is the age of a knowledge-base entry and  $\lambda$  controls the decay rate;  $\lambda$  would need to be tuned per technique category, since some techniques (e.g., long-lived living-off-the-land binaries) remain relevant far longer than others (e.g., short-lived C2 infrastructure).

**Confidence calibration.** Following standard temperature scaling:

$$P_{\text{correct}} = \sigma\left(\frac{z}{T}\right) = \frac{1}{1 + e^{-z/T}} \quad (3)$$

where  $z$  is the model’s raw output logit/score and  $T$  is a temperature parameter fit on a held-out calibration set.

**Hybrid retrieval.** We propose combining dense embedding similarity with sparse lexical retrieval (e.g., BM25) to compensate for the well-documented brittleness of purely dense retrieval on rare technical terminology (CVE identifiers, malware family names) that embeddings may not represent well.

### 3.6 Adversarial Defense Mechanisms

Given that Extractor Agent inputs are, by design, adversary-influenced text (OSINT, social media, and potentially attacker-planted decoy reporting), we treat prompt-injection and RAG-poisoning resistance as a first-class design requirement rather than an afterthought, informed directly by the current attack literature [16,17,18]:

- **Input/instruction separation.** Retrieved and ingested text is architecturally separated from system instructions (type-directed privilege separation [17]), rather than concatenated into a single undifferentiated prompt.
- **Cross-agent consistency checks.** Because the Mapper and Risk Assessor agents operate on the Extractor’s output rather than raw text, an injected instruction that survives extraction must additionally survive re-interpretation by a differently-prompted downstream agent – a defense-in-depth property analogous to the ”PALADIN” layered-defense proposal in the recent literature [17].
- **Confidence-gated human review.** Outputs below a calibrated confidence threshold (Equation 3) are routed to human review rather than auto-published, following common human-in-the-loop practice for high-stakes LLM deployments.
- **Retrieval corpus provenance filtering.** Following PoisonedRAG-style findings that a handful of adversarial documents can dominate retrieval [16], we propose weighting retrieval candidates by source provenance/reputation, not similarity alone.

We have designed but not yet implemented or red-teamed these mechanisms; Section 5 discusses this explicitly as a limitation.

## 4 Preliminary Experiment: Mapper Agent Retrieval Baseline

To ground at least one part of this architecture in real, reproducible measurement rather than projected performance, we implemented and evaluated a non-LLM baseline for the Mapper Agent’s core retrieval sub-task described in Section 3.3: given a short free-text description of adversary behaviour, rank MITRE ATT&CK techniques by relevance.



**Table 1.** Preliminary evaluation setup for the Mapper Agent retrieval baseline.

| Component                                   | Value                                        |
|---------------------------------------------|----------------------------------------------|
| ATT&CK version                              | Enterprise v18 (official MITRE STIX release) |
| Candidate techniques (top-level)            | 222                                          |
| Candidate techniques (incl. sub-techniques) | 697                                          |
| Hand-labelled evaluation sentences          | 49                                           |
| Annotators                                  | 1 (author-labelled; single-annotator)        |
| Label granularity                           | Top-level technique                          |

#### 4.1 Data

**Technique catalogue.** We downloaded the official MITRE ATT&CK Enterprise dataset (STIX 2.1 bundle, version 18) directly from MITRE’s public GitHub repository. After filtering revoked and deprecated objects, the catalogue contains 222 top-level techniques and 475 sub-techniques; for this baseline we restrict candidates to the 222 top-level techniques to keep the label space aligned with the granularity of our evaluation sentences.

**Evaluation set.** We manually authored 49 short (one-sentence) CTI-style behavioural descriptions, each hand-labelled by the authors with the single correct top-level ATT&CK technique, drawing on well-documented, publicly known adversary behaviours (e.g., credential dumping via Mimikatz, scheduled-task persistence, DNS-tunnelled C2, Kerberos ticket abuse). We emphasise that this is a small, illustrative, single-annotator set intended to produce one honest reference data point for this paper – not a validated benchmark; Section 5 discusses this limitation and the larger, multi-annotator evaluation required before any claims of general mapping accuracy would be warranted.

Table 1 summarises the resulting evaluation setup.

#### 4.2 Method

We build a TF-IDF vector representation (unigrams and bigrams, English stop-word removal) of each technique’s name and official MITRE description, and represent each evaluation sentence in the same vector space. For each sentence, we rank all 222 candidate techniques by cosine similarity and record the rank of the true label. This mirrors, in a deliberately simplified non-LLM form, the retrieval step at the core of RAG-based mapping systems such as TechniqueRAG [4], and serves as a “Mapper Agent without LLM reasoning” reference point analogous to an ablation baseline.

#### 4.3 Metrics

We report Top-1 and Top-3 accuracy, and Mean Reciprocal Rank:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i} \quad (4)$$

**Table 2.** Preliminary Mapper Agent retrieval results (real, measured;  $N = 49$  hand-labelled sentences, 222 candidate top-level techniques). Random-baseline values are the closed-form expectation under uniform random ranking, not a simulated run.

| Method                                         | Top-1 Acc.               | Top-3 Acc.   | MRR          |
|------------------------------------------------|--------------------------|--------------|--------------|
| Random ranking (analytic expectation)          | 0.004                    | 0.014        | 0.027        |
| TF-IDF cosine-similarity retrieval (this work) | <b>0.531</b>             | <b>0.714</b> | <b>0.655</b> |
| LLM-based Mapper Agent (proposed, Section 3.3) | <i>not yet evaluated</i> | –            | –            |
| Full ThreatGPT pipeline (proposed)             | <i>not yet evaluated</i> | –            | –            |

where  $\text{rank}_i$  is the rank position of the correct technique for the  $i$ -th evaluation sentence, and  $N = 49$ .

#### 4.4 Results

Table 2 reports the real, measured results of this baseline, alongside the analytically-computed expected performance of a random ranking over the same 222-technique candidate set.

The TF-IDF baseline correctly ranks the true technique first for just over half of the evaluation sentences (26/49) and within the top three for nearly three-quarters (35/49), a very large improvement over random ranking (MRR 0.655 vs. 0.027). Inspection of the errors is informative: several of the baseline’s failures are cases where the correct technique is a specific, low-frequency term (e.g., “DNS tunneling” mapped to the broader parent technique *Application Layer Protocol* rather than being disambiguated from adjacent techniques like *Exfiltration Over C2 Channel*), and cases where the sentence describes an “effect” of an action (e.g., “enumerated all user accounts”) using vocabulary that does not lexically overlap with the official ATT&CK description text even though the semantic mapping is clear to a human analyst. Both failure modes are exactly what dense, semantically-aware LLM embeddings and generative re-ranking – rather than lexical TF-IDF overlap – are designed to address, which is the specific, falsifiable hypothesis a full evaluation of the proposed Mapper Agent should test.

#### 4.5 What This Result Does and Does Not Show

This experiment demonstrates that (a) our evaluation harness, real ATT&CK data pipeline, and metric implementation are correct and reproducible, and (b) even a simple lexical baseline achieves non-trivial mapping accuracy, giving a concrete floor for comparison. It does *not* demonstrate that the proposed LLM-based Mapper Agent, the Extractor Agent, the Risk Assessor Agent, the temporal-weighting mechanism, or the adversarial defenses outperform this or any other baseline, because none of those components have yet been implemented and run. We view closing this gap as the primary empirical contribution of a follow-on study, for which this paper establishes the data pipeline, evaluation set, and first reference point.

## 5 Discussion

### 5.1 Practical Implications

If validated at full scale, an architecture along these lines could plausibly reduce SOC analyst triage time by structuring the extraction-mapping-assessment pipeline explicitly, integrating with existing SIEM/SOAR tooling via the structured intermediate objects each agent produces, and surfacing calibrated confidence scores so analysts can prioritise review effort. We deliberately do not quantify expected time savings or ROI here, since we have not deployed or user-tested the system.

### 5.2 Limitations

We are explicit about the current limitations of this work:

1. **No full implementation.** Only the Mapper Agent’s core retrieval sub-task has been prototyped, and only with a non-LLM baseline; the Extractor and Risk Assessor agents, the fine-tuning pipeline, the hierarchy-aware RAG index, and the adversarial defense layer are design proposals, not implemented and evaluated systems.
2. **Small, single-annotator evaluation set.** Our 49-sentence evaluation set was authored and labelled by the paper’s authors, not independently annotated CTI analysts, and is far smaller than benchmarks such as CTIBench [9] or the TRAM corpus [20]. Results should be read as illustrative, not as a validated accuracy estimate for real-world CTI text.
3. **Top-level-only label granularity.** Restricting to 222 top-level techniques rather than the full 697-technique catalogue (including sub-techniques) makes the task easier than the full ATT&CK mapping problem SOC analysts actually face.
4. **Domain and language boundaries.** All proposed components are designed and tested (where tested at all) on English-language Enterprise ATT&CK content; behaviour on ICS/Mobile matrices or non-English CTI sources is untested.
5. **Adversarial robustness is unvalidated.** The defense mechanisms in Section 3.6 are motivated by the published attack literature but have not been red-teamed against this specific architecture.
6. **Cost and latency of the full pipeline are unknown** until an end-to-end implementation exists to measure.

### 5.3 Ethical Considerations

An automated CTI system of this kind raises several considerations that deployers should weigh explicitly. First, the Extractor Agent processes third-party and potentially personal data present in OSINT and social-media text, raising data-protection questions (e.g., under GDPR) about retention and onward use. Second, LLM outputs can be miscalibrated or biased toward well-represented threat

actors and English-language reporting, which could systematically under-serve analysis of threats reported primarily in other languages. Third, an accurate, automated TTP-mapping and risk-scoring capability is inherently dual-use: the same extraction and reasoning capability that helps defenders triage threats faster could, in principle, help an attacker understand which of their own techniques are best documented or most likely to be flagged. We believe transparency about capabilities and limitations, human-in-the-loop review for high-stakes outputs, and restricting deployment to defensive contexts are appropriate mitigations, consistent with current guidance in the LLM-security literature [17].

## 6 Related Work

Table 3 qualitatively situates ThreatGPT relative to representative recent systems across the categories most relevant to our architecture. We deliberately omit specific performance figures for other systems in this table: reported metrics in the literature use different datasets, label granularities, and train/test splits, and repeating them out of context without re-running the systems ourselves would risk implying comparability that does not exist.

Relative to this landscape, ThreatGPT’s distinguishing design choices are (i) explicit separation of extraction, mapping, and organisational risk assessment into differently-prompted agents rather than a single model or a single-purpose tool, (ii) a knowledge layer designed for temporal recency and taxonomy hierarchy jointly, rather than either alone, and (iii) adversarial defense as an architectural cross-cutting concern rather than a downstream filter. We reiterate that these are design contributions, validated so far only for the retrieval sub-component reported in Section 4.

## 7 Conclusion and Future Work

### 7.1 Summary

We presented ThreatGPT, a hierarchical multi-agent architecture for LLM-based cyber threat intelligence analysis, decomposing the task into extraction, ATT&CK mapping, and organisational risk assessment, coordinated by a temporally-weighted, hierarchy-aware retrieval layer with adversarial-robustness safeguards designed in from the outset. Rather than reporting speculative end-to-end results for an unbuilt system, we implemented and evaluated a real, reproducible non-LLM baseline for the Mapper Agent’s core retrieval sub-task against the official MITRE ATT&CK Enterprise catalogue, obtaining 53.1% top-1 accuracy and an MRR of 0.655 against a hand-labelled evaluation set – a concrete floor for future comparison, released alongside this paper.

### 7.2 Future Work

We identify five concrete next steps:

**Table 3.** Qualitative comparison with representative related systems (categories, not reproduced performance numbers).

| System                       | Year    | Core approach                             | Multi-agent? | Notes                                                                                                          |
|------------------------------|---------|-------------------------------------------|--------------|----------------------------------------------------------------------------------------------------------------|
| TRAM [20]                    | 2020/23 | Supervised classifier + human review tool | No           | MITRE Engenuity’s open-source ATT&CK mapping assistant                                                         |
| TechniqueRAG [4]             | 2025    | Flat dense-retrieval RAG                  | No           | State-of-the-art flat RAG for CTI→ATT&CK mapping                                                               |
| Hierarchical RAG [4]         | 2026    | Hierarchy-aware RAG                       | No           | Directly motivates our knowledge-layer design (Sec. 3.5)                                                       |
| IntelEx [5]                  | 2024    | LLM-driven attack-level extraction        | No           | Focused on extraction, not mapping or risk                                                                     |
| CTINexus [6]                 | 2024    | In-context-learning KG construction       | No           | Addresses data scarcity via optimised ICL                                                                      |
| AgentSOC [1]                 | 2026    | Multi-layer agent-SOC framework           | Yes          | Broader SOC scope; not CTI-mapping specific                                                                    |
| <b>ThreatGPT (this work)</b> | 2026    | Hierarchical 3-agent (extract/map/assess) | Yes          | Explicit agent specialisation + temporal RAG + adversarial defense design; preliminary retrieval baseline only |

1. **Implement and evaluate the LLM-based Mapper Agent** described in Section 3.3 against the same evaluation protocol as Section 4, to directly test whether dense semantic retrieval plus generative re-ranking improves on the TF-IDF floor reported here.
2. **Build and evaluate the Extractor and Risk Assessor Agents**, including fine-tuning the Extractor Agent on an annotated CTI corpus and validating Risk Assessor outputs against expert-assigned risk rankings.
3. **Expand the evaluation set** to a larger, multi-annotator, sub-technique-granularity benchmark, ideally aligned with or contributed back to existing efforts such as CTIBench [9].
4. **Red-team the proposed adversarial defenses** in Section 3.6 against realistic prompt-injection and RAG-poisoning attacks drawn from the current literature [16].
5. **Multi-lingual and cross-matrix extension**, testing the architecture on non-English CTI sources and on the ATT&CK ICS and Mobile matrices.

## Reproducibility

The MITRE ATT&CK Enterprise dataset used in Section 4 is publicly available from MITRE’s official GitHub repository (<https://github.com/mitre/cti>). Our evaluation sentences, labels, and TF-IDF baseline code will be released at <https://github.com/REPLACE-WITH-ORG/threatgpt> upon publication ([TODO: replace placeholder GitHub organisation before submission]).

## References

1. AgentSOC: A Multi-Layer Agentic AI Framework for Security Operations Automation. IEEE (2026). arXiv:2604.20134.
2. LLM-Enabled Multi-Agent Systems: Empirical Evaluation and Insights into Emerging Design Patterns & Paradigms. arXiv:2601.03328 (2026).
3. Anthropic: Mapping AI-enabled cyber threats. Anthropic Research (2026). <https://www.anthropic.com/research/attack-navigator>.
4. Hierarchical Retrieval Augmented Generation for Adversarial Technique Annotation in Cyber Threat Intelligence Text. arXiv:2604.14166 (2026).
5. Xu, M., Wang, H., Liu, J., Lin, Y., Liu, C., Lim, H.W., Dong, J.S.: IntelEx: A LLM-driven attack-level threat intelligence extraction framework. arXiv:2412.10872 (2024).
6. Cheng, Y., Bajaber, O., Tsegai, S.A., Song, D., Gao, P.: CTINexus: leveraging optimized LLM in-context learning for constructing cybersecurity knowledge graphs under data scarcity. arXiv:2410.21060 (2024).
7. Fayyazi, R., Taghdimi, R., Yang, S.J.: Advancing TTP analysis: Harnessing the power of large language models with retrieval augmented generation. In: 2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops), pp. 255–261. IEEE (2024).
8. Huang, Y.T., Vaitheeshwari, R., Chen, M.C., Lin, Y.D., Hwang, R.H., et al.: MITREtrieval: Retrieving MITRE techniques from unstructured threat reports by fusion of deep learning and ontology. IEEE Trans. Netw. Serv. Manag. 21(4), 4871–4887 (2024).
9. Alam, M.T., et al.: CTIBench: A benchmark for evaluating LLMs in cyber threat intelligence. In: Proc. NeurIPS 2024, Datasets and Benchmarks Track, pp. 50805–50825 (2024).
10. Rahman, M.R., et al.: What are the attackers doing now? Automating cyberthreat intelligence extraction from text on pace with the changing threat landscape: A survey. ACM Comput. Surv. 55(12), 1–36 (2023).
11. Rahman, M.R., Williams, L.: From threat reports to continuous threat intelligence: A comparison of attack technique extraction methods from textual artifacts. (2022).
12. MITRE ATT&CK: State of the Art and Way Forward. arXiv:2308.14016 (2023).
13. Comparing AI Agents to Cybersecurity Professionals in Real-World Penetration Testing. arXiv:2512.09882 (2025).
14. Secure autonomous cyber defense with LLM agents: A systematic review of autonomy, tool-augmented reasoning, and governance constraints. Computers & Electrical Engineering (ScienceDirect) (2026).

15. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proc. NeurIPS 2020, pp. 9459–9474 (2020).
16. PoisonedRAG: Knowledge poisoning attacks to retrieval-augmented generation of large language models. USENIX Security (2025).
17. Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, and Defense Mechanisms. MDPI Information/Electronics (2026).
18. OWASP: LLM01:2025 Prompt Injection – OWASP Gen AI Security Project. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (2025).
19. Wudali, P.N., et al.: Rule-ATT&CK Mapper (RAM): Mapping SIEM rules to TTPs using LLMs. arXiv:2502.02337 (2025).
20. Center for Threat-Informed Defense / MITRE Engenuity: TRAM: Threat Report ATT&CK Mapper. <https://github.com/center-for-threat-informed-defense/tram> (2020/2023).